cv_lite API参考手册

> 本手册来源为嘉楠科技官网，为方便大家查询，在此备份。由于版本迭代等原因，手册内的API不保证完全可用。所有经过我们测试的、确定可用的API方法，将以教程的形式，出现在本章的后续章节中

cv_lite API参考手册

API 介绍

grayscale_find_blobs
rgb888_find_blobs
grayscale_find_circles
rgb888_find_circles
grayscale_find_rectangles
rgb888_find_rectangles
grayscale_find_edges
rgb888_find_edges
grayscale_threshold_binary
rgb888_threshold_binary
rgb888_adjust_exposure
rgb888_adjust_exposure_fast
rgb888_white_balance_gray_world_fast
rgb888_white_balance_gray_world_fast_ex
rgb888_white_balance_white_patch
rgb888_white_balance_white_patch_ex
rgb888_erode
rgb888_dilate
rgb888_open
rgb888_close
rgb888_tophat
rgb888_blackhat
rgb888_gradient
rgb888_mean_blur
rgb888_gaussian_blur
rgb888_calc_histogram
rgb888_find_corners

API 介绍

grayscale_find_blobs

描述

在灰度图像中查找blob（连通区域），返回blob的位置信息。

语法

请保证Sensor配置的出图为灰度图，否则会导致错误。

```
import cv_lite

image_shape = [480,640]  # 高，宽

threshold = [230, 255]   # 二值化阈值范围（亮区域）/ Threshold range for binarization
min_area = 10            # 最小目标面积 / Minimum area to keep
kernel_size = 1          # 腐蚀核大小（可用于降噪）/ Erosion kernel size

img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取图像数据引用

# 执行灰度图二值连通域检测
blobs = cv_lite.grayscale_find_blobs(image_shape, img_np,threshold[0],
threshold[1],min_area, kernel_size)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
threshold_min	二值化阈值最小值，int类型	输入	
threshold_max	二值化阈值最大值，int类型	输入	
min_area	最小区域面积，int类型	输入	
kernel_size	核大小，int类型	输入	

返回值

返回值	描述
blobs	blob位置信息列表，每4个元素为一个blob的位置信息，包括位置x、y、w、h

rgb888_find_blobs

描述

在RGB888图像中查找blob（连通区域），返回blob的位置信息。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite

image_shape = [480,640]  # 高，宽

threshold = [120, 255, 0, 50, 0, 50]

min_area = 100    # 最小色块面积 / Minimum blob area
kernel_size = 1   # 腐蚀膨胀核大小（用于预处理）/ Kernel size for morphological ops

# 拍摄一帧图像 / Capture a frame
```

```
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用

# 调用 cv_lite 扩展进行色块检测，返回 [x, y, w, h, ...] 列表
blobs = cv_lite.rgb888_find_blobs(image_shape, img_np, threshold, min_area,
kernel_size)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
threshold	二值化阈值范围，list类型，包括R、G、B三个通道的阈值范围	输入	
min_area	最小区域面积，int类型	输入	
kernel_size	核大小，int类型	输入	

返回值

返回值	描述
blobs	blob位置信息列表，每4个元素为一个blob的位置信息，包括位置x、y、w、h

grayscale_find_circles

描述

在灰度图像中查找圆，返回圆的位置信息。

语法

请保证Sensor配置的出图为灰度图，否则会导致错误。

```
import cv_lite

image_shape = [480,640]  # 高，宽

# ------------------------------
# 霍夫圆检测参数 / Hough circle detection parameters
# ------------------------------
dp = 1            # 累加器分辨率比 / Inverse ratio of resolution
minDist = 20      # 圆心最小距离 / Minimum distance between centers
param1 = 80       # Canny 高阈值 / Upper threshold for Canny edge detector
param2 = 20       # 累加器阈值 / Accumulator threshold for center detection
minRadius = 10    # 最小圆半径 / Minimum radius to detect
maxRadius = 50    # 最大圆半径 / Maximum radius to detect

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取图像的 ndarray 引用 / Get image data reference
```

```
# 检测圆形 / Detect circles using Hough Transform
# 返回格式：[x1, y1, r1, x2, y2, r2, ...]
circles = cv_lite.grayscale_find_circles(image_shape, img_np,dp, minDist,param1,
param2,
minRadius, maxRadius)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
dp	累加器分辨率比，float类型	输入	
minDist	圆心最小距离，int类型	输入	
param1	Canny 高阈值，int类型	输入	
param2	累加器阈值，int类型	输入	
minRadius	最小圆半径，int类型	输入	
maxRadius	最大圆半径，int类型	输入	

返回值

返回值	描述
circles	圆信息列表，每3个元素为一个圆的信息，包括位置x、y、r

rgb888_find_circles

描述

在RGB888图像中查找圆，返回圆的位置信息。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite

image_shape = [480,640]  # 高，宽

# ------------------------------
# 霍夫圆检测参数 / Hough Circle parameters
# ------------------------------
dp = 1           # 累加器分辨率与图像分辨率的反比 / Inverse ratio of accumulator
resolution
minDist = 30     # 检测到的圆心最小距离 / Minimum distance between detected centers
param1 = 80      # Canny边缘检测高阈值 / Higher threshold for Canny edge detection
param2 = 20      # 霍夫变换圆心检测阈值 / Threshold for center detection in
accumulator
minRadius = 10   # 检测圆最小半径 / Minimum circle radius
maxRadius = 50   # 检测圆最大半径 / Maximum circle radius
```

```
# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用

# 调用 cv_lite 扩展的霍夫圆检测函数，返回圆参数列表 [x, y, r, ...]
circles = cv_lite.rgb888_find_circles(image_shape, img_np, dp, minDist, param1,
param2, minRadius, maxRadius)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
dp	累加器分辨率与图像分辨率的反比，float类型	输入	
minDist	检测到的圆心最小距离，int类型	输入	
param1	Canny边缘检测高阈值，int类型	输入	
param2	霍夫变换圆心检测阈值，int类型	输入	
minRadius	检测圆最小半径，int类型	输入	
maxRadius	检测圆最大半径，int类型	输入	

返回值

返回值	描述
circles	圆信息列表，每3个元素为一个圆的信息，包括位置x、y、r

grayscale_find_rectangles

描述

在灰度图像中查找矩形，返回矩形的位置信息。

语法

请保证Sensor配置的出图为灰度图，否则会导致错误。

```
import cv_lite

image_shape = [480,640]  # 高，宽

# ------------------------------
# 矩形检测可调参数 / Adjustable rectangle detection parameters
# ------------------------------
canny_thresh1      = 50        # Canny 边缘检测低阈值 / Canny low threshold
canny_thresh2      = 150       # Canny 边缘检测高阈值 / Canny high threshold
approx_epsilon     = 0.04      # 多边形拟合精度比例（越小拟合越精确）/ Polygon
approximation accuracy
area_min_ratio     = 0.001     # 最小面积比例（相对于图像总面积）/ Min area ratio
```

```
max_angle_cos      = 0.3        # 最大角度余弦（越小越接近矩形）/ Max cosine of angle
between edges
gaussian_blur_size = 5          # 高斯模糊核尺寸（奇数）/ Gaussian blur kernel size

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()

# 调用底层矩形检测函数
# 返回格式：[x0, y0, w0, h0, x1, y1, w1, h1, ...]
rects = cv_lite.grayscale_find_rectangles(
    image_shape, img_np,
    canny_thresh1, canny_thresh2,
    approx_epsilon,
    area_min_ratio,
    max_angle_cos,
    gaussian_blur_size
)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
canny_thresh1	Canny 边缘检测低阈值，int类型	输入	
canny_thresh2	Canny 边缘检测高阈值，int类型	输入	
approx_epsilon	多边形拟合精度比例，float类型	输入	
area_min_ratio	最小面积比例，float类型	输入	
max_angle_cos	最大角度余弦，float类型	输入	
gaussian_blur_size	高斯模糊核尺寸，int类型	输入	

返回值

返回值	描述
rects	矩形位置信息列表，每4个元素为一个矩形的位置信息，包括位置x、y、w、h

rgb888_find_rectangles

描述

在RGB888图像中查找矩形，返回矩形的位置信息。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
```

```
import cv_lite

image_shape = [480,640]  # 高，宽

# ------------------------------
# 可调参数（建议调试时调整）/ Adjustable parameters (recommended for tuning)
# ------------------------------
canny_thresh1       = 50        # Canny 边缘检测低阈值 / Canny edge low threshold
canny_thresh2       = 150       # Canny 边缘检测高阈值 / Canny edge high threshold
approx_epsilon      = 0.04      # 多边形拟合精度（比例） / Polygon approximation
precision (ratio)
area_min_ratio      = 0.001     # 最小面积比例（0~1） / Minimum area ratio (0~1)
max_angle_cos       = 0.5       # 最大角余弦（值越小越接近矩形） / Max cosine of angle
(smaller closer to rectangle)
gaussian_blur_size  = 5         # 高斯模糊核大小（奇数） / Gaussian blur kernel size
(odd number)

# 拍摄当前帧图像 / Capture current frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference

# 调用底层矩形检测函数，返回矩形列表 [x0, y0, w0, h0, x1, y1, w1, h1, ...]
# Call underlying rectangle detection function, returns list of rectangles [x, y,
w, h, ...]
rects = cv_lite.rgb888_find_rectangles(
    image_shape, img_np,
    canny_thresh1, canny_thresh2,
    approx_epsilon,
    area_min_ratio,
    max_angle_cos,
    gaussian_blur_size
)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
canny_thresh1	Canny 边缘检测低阈值，int类型	输入	
canny_thresh2	Canny 边缘检测高阈值，int类型	输入	
approx_epsilon	多边形拟合精度比例，float类型	输入	
area_min_ratio	最小面积比例，float类型	输入	
max_angle_cos	最大角度余弦，float类型	输入	
gaussian_blur_size	高斯模糊核尺寸，int类型	输入	

返回值

返回值	描述
rects	矩形位置信息列表，每4个元素为一个矩形的位置信息，包括位置x、y、w、h

grayscale_find_edges

描述

在灰度图像中查找边缘，返回边缘检测的图像ulab.numpy.ndarray数据,可在返回数据基础上创建image实例。

语法

请保证Sensor配置的出图为灰度图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# ------------------------------
# 边缘检测参数 / Canny edge detection thresholds
# ------------------------------
threshold1 = 50  # 低阈值 / Lower threshold
threshold2 = 80  # 高阈值 / Upper threshold

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 ndarray 引用 / Get ndarray reference

# 调用 cv_lite 扩展执行边缘检测 / Perform edge detection
# 返回灰度边缘图像 ndarray / Returns edge image ndarray
edge_np = cv_lite.grayscale_find_edges(
    image_shape, img_np, threshold1, threshold2)

# 包装边缘图像用于显示 / Wrap ndarray as image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=edge_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
threshold1	低阈值，int类型	输入	
threshold2	高阈值，int类型	输入	

返回值

返回值	描述
edge_np	边缘检测后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_find_edges

描述

在RGB888图像中查找边缘，返回边缘检测的ulab.numpy.ndarray图像数据，可在此数据上创建image实例。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# ------------------------------
# 边缘检测参数 / Canny edge detection thresholds
# ------------------------------
threshold1 = 50  # 低阈值 / Lower threshold
threshold2 = 80  # 高阈值 / Upper threshold

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用

# 调用 cv_lite 扩展的边缘检测函数，返回灰度边缘图 ndarray
edge_np = cv_lite.rgb888_find_edges(image_shape, img_np, threshold1, threshold2)

# 构造灰度图像对象用于显示
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
alloc=image.ALLOC_REF, data=edge_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
threshold1	低阈值，int类型	输入	
threshold2	高阈值，int类型	输入	

返回值

返回值	描述
edge_np	边缘检测后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

grayscale_threshold_binary

描述

在灰度图像中进行二值化处理，返回二值化后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例。

语法

请保证Sensor配置的出图为灰度图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# ------------------------------
# 二值化参数设置 / Binary threshold parameters
# ------------------------------
thresh = 130   # 阈值 / Threshold value
maxval = 255   # 最大值，二值化后白色像素值 / Max value for white pixels

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 ndarray 引用 / Get ndarray reference

# 调用 cv_lite 扩展进行二值化处理
# 返回二值化后的灰度图 ndarray / Returns binary image ndarray
binary_np = cv_lite.grayscale_threshold_binary(image_shape, img_np, thresh,
maxval)

# 构造用于显示的灰度图像对象 / Wrap ndarray as grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=binary_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
thresh	阈值，int类型	输入	
maxval	最大值，int类型	输入	

返回值

返回值	描述
binary_np	二值化后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_threshold_binary

描述

在RGB888图像中进行二值化处理，返回二值化后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# --------------------------------
# 二值化参数设置 / Binary threshold parameters
# --------------------------------
thresh = 130   # 阈值 / Threshold value
maxval = 255   # 最大值，二值化后白色像素值 / Max value for white pixels

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 ndarray 引用 / Get ndarray reference

# 调用二值化接口（返回 ndarray）/ Call binary threshold function (returns ndarray)
binary_np = cv_lite.rgb888_threshold_binary(image_shape, img_np, thresh, maxval)

# 构造灰度图像用于显示 / Construct grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=binary_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
thresh	阈值，int类型	输入	
maxval	最大值，int类型	输入	

返回值

返回值	描述
binary_np	二值化后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_adjust_exposure

描述

在RGB888图像中调整曝光度，返回调整后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# --------------------------------
# 曝光增益因子（<1 降低亮度，>1 增加亮度）/ Exposure gain factor
# --------------------------------
exposure_gain = 2.5          # 推荐范围 0.2 ~ 3.0；1.0 表示无增益 / Recommended
range: 0.2~3.0, 1.0 = no gain

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像数据引用 / Get RGB888 ndarray
reference (HWC)

# 调用 cv_lite 模块进行曝光调节 / Apply exposure adjustment using cv_lite module
exposed_np = cv_lite.rgb888_adjust_exposure(image_shape, img_np, exposure_gain)

# 包装图像用于显示 / Wrap processed image for display
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=exposed_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
exposure_gain	曝光增益因子，float类型	输入	

返回值

返回值	描述
exposed_np	调整曝光度后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_adjust_exposure_fast

描述

在RGB888图像中快速调整曝光度，返回调整后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。该方法是上面使用软件曝光的加速版本。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# --------------------------------
# 曝光调节参数 / Exposure gain parameter
# exposure_gain: 曝光增益因子，范围建议 0.2 ~ 3.0
# 小于 1.0 为减弱曝光（变暗），大于 1.0 为增强曝光（变亮）
# exposure_gain < 1.0: darker, > 1.0: brighter
# --------------------------------
exposure_gain = 2.5  # 示例：增强亮度 1.5 倍 / Example: brighten image by 1.5x

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray (HWC)

# 使用 cv_lite 模块进行曝光调整 / Apply exposure gain using cv_lite
exposed_np = cv_lite.rgb888_adjust_exposure_fast(
    image_shape,
    img_np,
    exposure_gain
)

# 构造图像用于显示 / Wrap processed image for display
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=exposed_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
exposure_gain	曝光增益因子，float类型	输入	

返回值

返回值	描述
exposed_np	调整曝光度后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_white_balance_gray_world_fast

描述

在RGB888图像中使用灰度世界算法快速进行白平衡，返回白平衡后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。该方法是上面使用软件白平衡的加速版本。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray reference (HWC)

# 使用 cv_lite 进行加速灰度世界白平衡处理
# Apply fast gray world white balance
balanced_np = cv_lite.rgb888_white_balance_gray_world_fast(image_shape, img_np)

# 包装图像用于显示 / Wrap processed image for display
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,alloc=image.ALLOC_REF, data=balanced_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	

返回值

返回值	描述
balanced_np	白平衡后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_white_balance_gray_world_fast_ex

描述

在RGB888图像中使用灰度世界算法快速进行白平衡，返回白平衡后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。该方法是上面方法的参数可调版本。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image
```

```
image_shape = [480,640]  # 高，宽

# ------------------------------
# 设置白平衡参数 / White balance parameters
# ------------------------------
gain_clip = 2.5             # 增益限制系数，防止偏色 / Gain limit to prevent color
blowout
brightness_boost = 1.25     # 亮度增强系数 / Global brightness boost

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray reference
(HWC)

# 使用 cv_lite 进行加速灰度世界白平衡处理（可调参数）
# Apply fast gray world white balance with tunable parameters
balanced_np = cv_lite.rgb888_white_balance_gray_world_fast_ex(
    image_shape,
    img_np,
    gain_clip,
    brightness_boost
)

# 包装图像用于显示 / Wrap processed image for display
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=balanced_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
gain_clip	增益限制系数，float类型	输入	
brightness_boost	亮度增强系数，float类型	输入	

返回值

返回值	描述
balanced_np	白平衡后的图像数据，ulab.numpy.ndarray类型，可在此数据上创建image实例用于显示

rgb888_white_balance_white_patch

描述

在RGB888图像中使用白色补丁算法进行白平衡，返回白平衡后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# 拍摄一帧图像
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # RGB888 原始图像（ulab ndarray）

# 调用 cv_lite 扩展模块进行白色补丁白平衡
balanced_np = cv_lite.rgb888_white_balance_white_patch(image_shape, img_np)

# 构造 RGB888 显示图像
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=balanced_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	

返回值

返回值	描述
balanced_np	白平衡后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例

rgb888_white_balance_white_patch_ex

描述

在RGB888图像中使用白色补丁算法进行白平衡，返回白平衡后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。该方法是上面方法的参数可调版本。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640]  # 高，宽

# ------------------------------
# 白平衡参数 / White balance config
# ------------------------------
top_percent = 5.0          # 用于白点估计的最亮像素百分比 / Top N% brightest pixels
used for white patch
gain_clip = 2.5            # 最大增益限制 / Limit gain to avoid over-brightening
brightness_boost = 1.1     # 提亮因子 / Global brightness scaling

# 拍摄一帧图像 / Capture one frame
img = sensor.snapshot()
```

```
img_np = img.to_numpy_ref()  # 获取图像数据引用（ulab ndarray）

# 执行白点白平衡处理 / Apply white patch white balance with parameters
balanced_np = cv_lite.rgb888_white_balance_white_patch_ex(
    image_shape, img_np,
    top_percent,
    gain_clip,
    brightness_boost
)

# 包装图像用于显示 / Wrap balanced image as displayable image object
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=balanced_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
top_percent	用于白点估计的最亮像素百分比，float类型	输入	
gain_clip	最大增益限制，float类型	输入	
brightness_boost	提亮因子，float类型	输入	

返回值

返回值	描述
balanced_np	白平衡后的图像数据，ulab.numpy.ndarray类型，在此数据上创建image实例用于显示

rgb888_erode

描述

在RGB888图像中进行腐蚀操作，返回腐蚀后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite

# ---------------------------------
# 腐蚀算法参数设置 / Erosion parameters
# ---------------------------------
kernel_size = 3          # 卷积核尺寸（必须为奇数，如 3, 5, 7）/ Kernel size (must be odd)
iterations = 1           # 腐蚀迭代次数 / Number of erosion passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for binarization (0 = Otsu)
```

```
# 获取一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference (HWC)

# 应用腐蚀操作（自动转换为灰度后进行） / Apply erosion (converts RGB to gray
internally)
eroded_np = cv_lite.rgb888_erode(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造图像用于显示（灰度格式） / Wrap eroded grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=eroded_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	腐蚀迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
eroded_np	腐蚀后的图像数据，ulab.numpy.ndarray类型

rgb888_dilate

描述

在RGB888图像中进行膨胀操作，返回膨胀后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# ------------------------------
```

```
# 膨胀算法参数设置 / Dilation parameters
# ------------------------------
kernel_size = 3          # 卷积核尺寸 / Kernel size (推荐为奇数，但部分实现支持偶数)
iterations = 1           # 膨胀迭代次数 / Number of dilation passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)

# 获取一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference

# 应用膨胀操作（内部先转换为灰度再膨胀）/ Apply dilation (converts RGB to gray
internally)
result_np = cv_lite.rgb888_dilate(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造图像用于显示（灰度格式） / Wrap dilated grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=result_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	膨胀迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
dilated_np	膨胀后的图像数据，ulab.numpy.ndarray类型

rgb888_open

描述

对RGB888图像进行开运算，返回开运算后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# --------------------------------
# 开操作参数设置 / Opening parameters
# --------------------------------
kernel_size = 3          # 卷积核尺寸（应为奇数）/ Kernel size (should be odd)
iterations = 1           # 操作迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference (HWC)

# 执行开操作（先腐蚀再膨胀）/ Apply opening (erode then dilate)
open_np = cv_lite.rgb888_open(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造灰度图像对象用于显示 / Wrap processed grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=open_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	开运算迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
opened_np	开运算后的图像数据，ulab.numpy.ndarray类型

rgb888_close

描述

对RGB888图像进行闭运算，返回闭运算后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# --------------------------------
# 闭操作参数设置 / Closing parameters
# --------------------------------
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 操作迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)

# 获取图像帧 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference (HWC)

# 应用闭操作（先膨胀后腐蚀）/ Apply closing (dilate then erode)
closed_np = cv_lite.rgb888_close(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造图像对象（灰度图）用于显示 / Wrap processed grayscale image for display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=closed_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	闭运算迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
closed_np	闭运算后的图像数据，ulab.numpy.ndarray类型

rgb888_tophat

描述

对RGB888图像进行顶帽运算，返回顶帽运算后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# ------------------------------
# 顶帽运算参数设置 / Top-Hat parameters
# ------------------------------
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 运算迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)

# 获取一帧图像并转换为 ndarray / Capture a frame and convert to ndarray
img = sensor.snapshot()
img_np = img.to_numpy_ref()

# 执行顶帽运算 / Apply Top-Hat operation
# 顶帽 = 原图 - 开运算结果 / Top-Hat = Original - Opening
tophat_np = cv_lite.rgb888_tophat(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造图像对象并显示 / Wrap result as image and display
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=tophat_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	顶帽运算迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
tophat_np	顶帽运算后的图像数据，ulab.numpy.ndarray类型

rgb888_blackhat

描述

对RGB888图像进行黑帽运算，返回黑帽运算后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# --------------------------------
# 黑帽运算参数设置 / Black-Hat parameters
# --------------------------------
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 运算迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)

# 获取一帧图像并转换为 ndarray / Capture a frame and convert to ndarray
img = sensor.snapshot()
img_np = img.to_numpy_ref()

# 执行黑帽运算 / Apply Black-Hat operation
# 黑帽 = 闭运算 - 原图 / Black-Hat = Closing - Original
blackhat_np = cv_lite.rgb888_blackhat(
    image_shape,
    img_np,
    kernel_size,
    iterations,
    threshold_value
)

# 构造图像对象并显示 / Wrap result as image and display
```

```
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=blackhat_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	黑帽运算迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
blackhat_np	黑帽运算后的图像数据，ulab.numpy.ndarray类型

rgb888_gradient

描述

对RGB888图像进行形态学梯度运算，返回形态学梯度后的单通道ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# ==================================
# 梯度操作参数 / Morphological Gradient parameters
# ==================================
kernel_size = 3         # 卷积核尺寸（建议奇数）/ Kernel size (recommended odd)
iterations = 1          # 形态学迭代次数 / Morphology iterations
threshold_value = 100   # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold value (0=use
Otsu)

# 获取图像并转为 ndarray / Capture image and convert to ndarray
img = sensor.snapshot()
img_np = img.to_numpy_ref()

# 调用梯度操作（Gradient = 膨胀 - 腐蚀）/ Call morphological gradient (dilate -
erode)
gradient_np = cv_lite.rgb888_gradient(image_shape, img_np, kernel_size,
iterations, threshold_value)

# 构造灰度图像用于显示 / Construct grayscale image for display
```

```
img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=gradient_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	
iterations	形态学迭代次数，int类型	输入	
threshold_value	二值化阈值，int类型	输入	

返回值

返回值	描述
gradient_np	形态学梯度后的图像数据，ulab.numpy.ndarray类型

rgb888_mean_blur

描述

对RGB888图像进行均值模糊，返回模糊后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# --------------------------------
# 均值模糊核大小 / Mean blur kernel size
# 必须为奇数，例如 3 / 5 / 7 / Must be odd: 3, 5, 7, etc.
# --------------------------------
kernel_size = 3

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray (HWC)

# 应用均值模糊滤波 / Apply mean blur using cv_lite
mean_blur_np = cv_lite.rgb888_mean_blur_fast(
    image_shape,
    img_np,
    kernel_size
)

# 构造图像用于显示 / Wrap processed image for display
```

```
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=mean_blur_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	

返回值

返回值	描述
mean_blur_np	均值模糊后的图像数据，ulab.numpy.ndarray类型

rgb888_gaussian_blur

描述

对RGB888图像进行高斯模糊，返回模糊后的ulab.numpy.ndarray图像数据，可在此数据上创建image实例用于显示。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite
import image

image_shape = [480,640] # 高，宽

# ------------------------------
# 高斯滤波核大小 / Gaussian blur kernel size
# 必须为奇数，例如 3 / 5 / 7 / Must be odd: 3, 5, 7, etc.
# ------------------------------
kernel_size = 3

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray (HWC)

# 应用高斯模糊滤波 / Apply gaussian blur using cv_lite
gaussian_blur_np = cv_lite.rgb888_gaussian_blur_fast(
    image_shape,
    img_np,
    kernel_size
)

# 构造图像用于显示 / Wrap processed image for display
img_out = image.Image(image_shape[1], image_shape[0], image.RGB888,
                      alloc=image.ALLOC_REF, data=gaussian_blur_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，顺序为[高，宽]，如[480,640]	输入	
img_np	图像数据引用，ulab.numpy.ndarray类型	输入	
kernel_size	卷积核尺寸，int类型	输入	

返回值

返回值	描述
gaussian_blur_np	高斯模糊后的图像数据，ulab.numpy.ndarray类型

rgb888_calc_histogram

描述

计算RGB888图像的直方图，返回直方图数据。

语法

```
import cv_lite

# 拍摄一帧图像 / Capture a frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 图像引用 / Get RGB888 ndarray reference (HWC)

# 使用 cv_lite 计算 RGB 直方图（返回 shape 为 3x256 的数组）
# Calculate RGB888 histogram using cv_lite (3x256 array)
hist = cv_lite.rgb888_calc_histogram(image_shape, img_np)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，包括宽高，如[1920,1080]	输入	
img_np	图像数据引用	输入	

返回值

返回值	描述
hist	直方图数据

rgb888_find_corners

描述

在RGB888图像中查找图像中的角点，返回角点的坐标。

语法

请保证Sensor配置的出图为RGB888图，否则会导致错误。

```
import cv_lite

image_shape = [240,320] # 高，宽

# ------------------------------
# 可调参数（建议调试时调整）/ Adjustable parameters (recommended for tuning)
# ------------------------------
max_corners       = 20        # 最大角点数 / Maximum number of corners
quality_level     = 0.01      # Shi-Tomasi质量因子 / Corner quality factor (0.01 ~
0.1)
min_distance      = 20.0      # 最小角点距离 / Minimum distance between corners

# 拍摄当前帧图像 / Capture current frame
img = sensor.snapshot()
img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference

# 调用角点检测函数，返回角点数组 [x0, y0, x1, y1, ...]
corners = cv_lite.rgb888_find_corners(
    image_shape, img_np,
    max_corners,
    quality_level,
    min_distance
)

# 遍历角点数组，绘制角点 / Draw detected corners
for i in range(0, len(corners), 2):
    x = corners[i]
    y = corners[i + 1]
    img.draw_circle(x, y, 3, color=(0, 255, 0), fill=True)
```

参数

参数名称	描述	输入 / 输出	说明
image_shape	图像形状，list类型，包括宽高，如[240,320]	输入	
img_np	图像数据引用	输入	
max_corners	最大角点数	输入	
quality_level	Shi-Tomasi质量因子	输入	
min_distance	最小距离	输入	

返回值

返回值	描述
corners	角点坐标数组，每两个数表示一个角点的坐标，如[x0,y0,x1,y1,...]